

State Management in React

Von Basics bis zum globalen State



Was ist State?

Daten, die sich zur Laufzeit ändern



Counter – die Zahl ändert sich bei jedem Klick



Suchfeld – der Text ändert sich beim Tippen



Login – eingeloggt oder nicht



Modal – offen oder geschlossen

Level

1 Lokal

Eine Komponente

2 Global

Mehrere Komponenten

3 App-weit

Externe Libraries

useState

Lokaler State für eine Komponente

```
const [count, setCount] = useState(0);
```

//
//

↑
Wert

↑
Setter

↑
Startwert

Wert lesen

count ist immer aktuell

Wert ändern

Nur über setCount()

Lokal

Nur in dieser Komponente

Counter - Live Coding



```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  function increment() {
    setCount(count + 1)
  }
  function decrement() { setCount(count - 1) }
  function reset() { setCount(0) }

  return (
    <div>
      <h2>Zähler: {count}</h2>
      <button onClick={increment}>+1</button>
    </div>
  );
}
```

Custom Hook

State-Logik auslagern → sauberer Code

Vorher

```
function Counter() {  
  const [count, setCount]  
    = useState(0);  
  
  function increment() {...}  
  function decrement() {...}  
  function reset() {...}  
  
  return ( /* JSX */ );  
}
```



Nachher

```
function Counter() {  
  const { count,  
    increment, decrement,  
    reset } = useCounter();  
  
  return ( /* nur JSX */ );  
}
```

Logik & UI getrennt • Wiederverwendbar • Einfach testbar

useCounter - Custom Hook



```
import { useState } from "react";

function useCounter(startValue = 0) {
  const [count, setCount] = useState(startValue);

  function increment() {
    setCount(count + 1);
  }
  function decrement() {
    setCount(count - 1);
  }
  function reset() { setCount(startValue); }

  return { count, increment, decrement, reset };
}
```

TodoList - Live Coding



```
const [todos, setTodos] = useState([]);
const [input, setInput] = useState("");

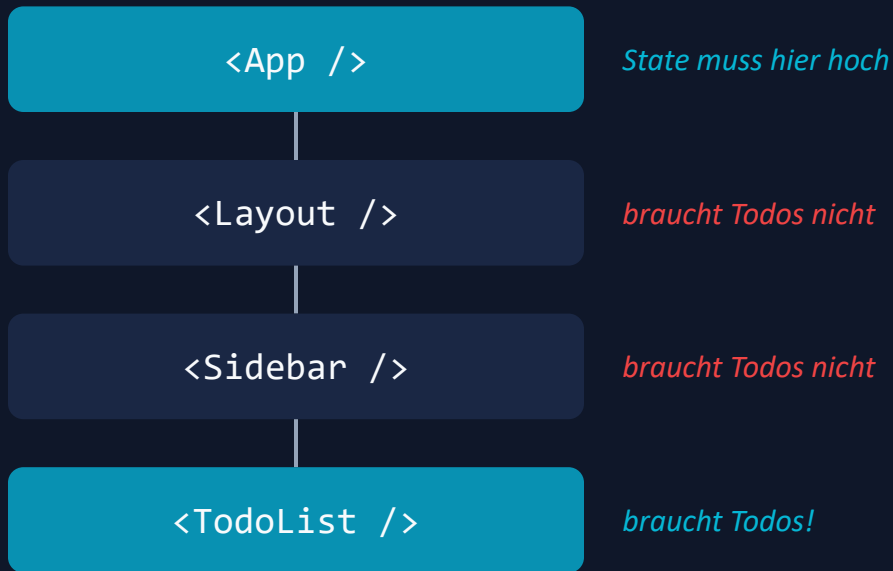
function addTodo() {
  if (input.trim() === "") return;
  setTodos([...todos,
    { id: Date.now(), text: input, done: false }]);
  setInput("");
}

function toggleTodo(id) {
  setTodos(todos.map(t =>
    t.id === id ? {...t, done: !t.done} : t));
}

function deleteTodo(id) {
  setTodos(todos.filter(t => t.id !== id));
}
```

Das Problem: Prop Drilling

Was wenn die Navbar die Todos braucht?



Context API

State global teilen – ohne Prop Drilling

1

Context erstellen

```
const ThemeCtx = createContext();
```

2

Provider wrappen

```
<ThemeCtx.Provider value={theme}>  
  <App />  
</ThemeCtx.Provider>
```

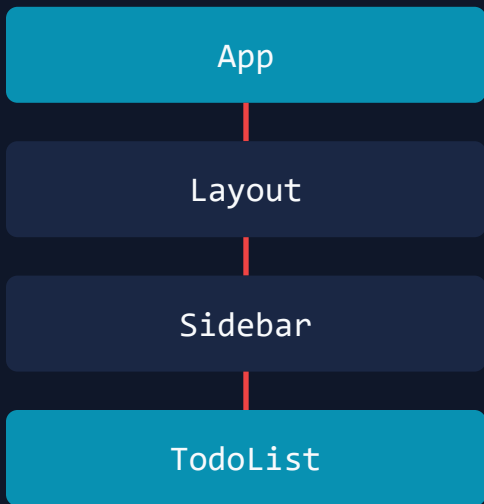
3

Überall konsumieren

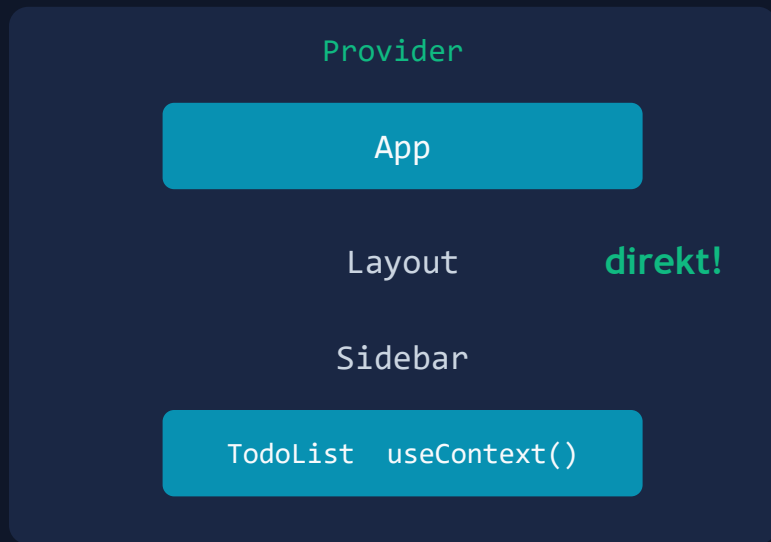
```
const theme = useContext(ThemeCtx);
```

Context vs. Prop Drilling

Prop Drilling



Context API



Context - Wann passt es?



Perfekt für

- Theme (Dark / Light Mode)
- Auth (eingeloggt / ausgeloggt)
- Sprache wechseln

Grenzen

- Re-Render: Alle Consumer bei jeder Änderung
- Nicht für schnelle Updates (Slider, Drag&Drop)
- Viele Contexts = "Provider Hell"

useReducer

Strukturierte State-Logik mit festen Regeln

State

Ein Objekt mit
allem Zustand



Action

Ein Zettel:
{ type: 'increment' }



Reducer

Regeln
(switch/case):
(state, action) =>
new

```
const [state, dispatch] = useReducer(reducer, initialState);
```

```
// Statt setCount(count + 1):  
dispatch({ type: 'increment' });  
// → Reducer entscheidet was passiert
```

useReducer - Counter

```
function reducer(state, action) {  
  switch (action.type) {  
    case 'increment':  
      return { count: state.count + 1 };  
    case 'decrement':  
      return { count: state.count - 1 };  
    case 'reset':  
      return { count: 0 };  
    default: return state;  
  }  
}  
  
const [state, dispatch] = useReducer(reducer, { count: 0 });  
  
<button onClick={() => dispatch({ type: 'increment' })}>  
  +1  
</button>
```

Redux

= *useReducer auf App-Ebene*

useReducer (lokal)

```
function reducer(state, action) {  
  switch (action.type) {  
    case 'increment':  
      return {count: state.count+1}  
  }  
}  
  
dispatch({ type: 'increment' })
```

Redux Toolkit (global)

```
const slice = createSlice({  
  reducers: {  
    increment: (state) => {  
      state.value += 1 }  
    }  
  })  
  
dispatch(increment())
```



Gleiche Konzepte (State, Action, Reducer) – Redux macht es global mit einem zentralen Store + Redux Toolkit reduziert Boilerplate.

Vergleich - Wann was?

	Scope	Ideal für
<code>useState</code>	Lokal	Einfache Werte (Counter, Input)
<code>Custom Hook</code>	Lokal	Logik auslagern & wiederverwenden
<code>Context</code>	Global	Theme, Auth, Sprache
<code>useReducer</code>	Lokal	Komplexe State-Logik
<code>Redux</code>	Global	Große Enterprise Apps



useState → Custom Hook → Context → useReducer → Redux

Zusammenfassung

- ✓ State = Daten die sich zur Laufzeit ändern
- ✓ useState für lokalen State in einer Komponente
- ✓ Custom Hooks für saubere, wiederverwendbare Logik
- ✓ Context API wenn mehrere Komponenten State teilen
- ✓ useReducer für strukturierte, komplexe State-Logik
- ✓ Redux = useReducer auf App-Ebene
- ✓ Starte einfach – skaliere bei Bedarf!



Fragen?

Danke fürs Zuhören!